

A Study on Pre-training, Fine-tuning, and RAG Strategies for Efficient Bug Fixing

CSCI 455/555: GenAI for SD

Antonio Mastropaolo

Nathaniel Callabresi

https://github.com/F4llow/GenAI_Assignment3/

Abstract

This report investigates the efficacy of domain-specific pre-training for small-scale T5 models in the context of Java bug fixing. We compare a T5-Small model pre-trained via span corruption (Pipeline A) against a model initialized from scratch (Pipeline B), as well as a Retrieval-Augmented Generation (RAG) approach using Qwen 1.5B. While Pipeline B achieved the highest raw Code-BLEU scores, Pipeline A demonstrated a **10.5x improvement in inference efficiency** over Pipeline B, proving that pre-training is critical for learning structural code constraints and efficient sequence termination. Ultimately, we explore the distinct deployment trade-offs between highly constrained, task-specific models and flexible, general-purpose LLMs.

1 Introduction

Automated program repair (APR) is a cornerstone of modern software engineering. While Large Language Models (LLMs) have set new benchmarks in APR, their deployment in resource-constrained environments is limited by size and latency. This study explores whether domain-specific pre-training on a smaller 60M-parameter architecture (T5-Small) can bridge the gap. We evaluate three distinct paradigms: (1) Pre-training then Fine-tuning (Pipeline A), (2) Fine-tuning from scratch (Pipeline B), and (3) Retrieval-Augmented Generation (RAG).

2 Tokenizer and Model Configuration

2.1 Tokenizer Training

To ensure a controlled experimental environment, a specialized tokenizer and model architecture were built entirely from scratch. A Unigram SentencePiece tokenizer was trained directly on the pre-training corpus of 50,000 Java methods extracted from CodeSearchNet. The Unigram algorithm was selected over BPE for its top-down pruning approach, which is highly effective for preserving whole-word code tokens. The vocabulary was limited to 16,384 tokens to balance subword granularity with the sparsity constraints of our 50K corpus. The design included 100 sentinel tokens (`<extra_id.0>` to `<extra_id.99>`) for the span corruption objective.

During development, we discovered a critical vulnerability in legacy Hugging Face `T5Tokenizer` wrappers, which failed to map subwords properly and collapsed the dataset into arrays of `<unk>` tokens. To resolve this, we utilized the Rust-based tokenizers library with a Metaspace pre-tokenizer and decoder, wrapped in a modern `PreTrainedTokenizerFast` to guarantee reliable parsing of Java whitespace.

2.2 Model Architecture

Following the T5-Small specification, we explicitly defined the configuration manually to avoid inheriting ambiguous checkpoint shapes. The configuration used for both Pipeline A and B is shown

below:

```
1 from transformers import T5Config, T5ForConditionalGeneration
2
3 t5_config = T5Config(
4     vocab_size=16384,           # Resized for custom tokenizer
5     decoder_start_token_id=0,  # Pad as start
6     pad_token_id=0,
7     eos_token_id=1,
8     d_model=512,               # Embedding dimension
9     d_ff=2048,                 # Feed-forward dimension
10    num_heads=8,                # Attention heads
11    num_layers=6,               # Encoder layers
12    num_decoder_layers=6        # Decoder layers
13 )
14 model = T5ForConditionalGeneration(config=t5_config)
15 model.resize_token_embeddings(len(tokenizer))
```

Listing 1: Manual T5-Small Configuration

3 Training Pipelines

3.1 Pre-training Setup (Pipeline A)

The pre-training objective utilized the **Span Corruption** task. 15% of tokens in the input sequence were masked and grouped into spans delimited by sentinel tokens. Training was executed on a Quadro RTX 6000. Because the Turing architecture lacks native BFloat16 support, standard mixed precision (`fp16=True`) resulted in catastrophic gradient explosions. We stabilized the run by enforcing full FP32 precision. The model was pre-trained for 3 epochs ($LR = 5e - 4$, Batch Size 32), with training loss decreasing from 3.31 to 2.81.

3.2 Fine-tuning Pipelines

- **Pipeline A (Pre-training + Fine-tuning):** The pre-trained model was fine-tuned on the CodeXGLUE Code Refinement (medium) dataset for 5 epochs.
- **Pipeline B (Fine-tuning From Scratch):** A second T5-small was initialized with identical, randomly assigned weights. It skipped the span-corruption pre-training entirely and was fine-tuned directly on the CodeXGLUE dataset for 5 epochs using identical hyperparameters ($LR = 1e - 4$, Batch Size=8).

4 Retrieval-Augmented Generation (RAG) Setup

To contrast task-specific small models against general-purpose LLMs, we evaluated **Qwen 2.5-Coder-1.5B-Instruct** using a RAG approach.

Retriever Design: We implemented a Dense Passage Retriever powered by `microsoft/codebert-base`. CodeBERT’s bimodal pre-training allows it to identify semantically analogous bug patterns rather than just surface-level structural syntax. The 52K CodeXGLUE training pairs were embedded via mean-pooling and stored in a FAISS index.

Prompting Strategy: Qwen was evaluated under a Zero-Shot setting and a 3-Shot RAG setting, where the top-3 retrieved bug-fix pairs were injected into the prompt.

5 Evaluation and Comparative Analysis

5.1 Results Summary

Evaluation was performed on the full 6,545 test samples. Table 1 summarizes the performance and runtime metrics.

Table 1: Final Results Comparison

Configuration	Exact Match (%)	CodeBLEU	Eval Runtime
Pipeline A (Pre-trained)	0.0000%	30.7260	0h 09m 53s
Pipeline B (Scratch)	0.0458%	51.9902	1h 43m 16s
Qwen 1.5B (Zero-Shot)	0.0000%	42.8432	0h 49m 23s
Qwen 1.5B (RAG 3-Shot)	0.0000%	44.5091	1h 00m 12s

5.2 Pre-training vs. From Scratch

Pipeline B (51.99 CodeBLEU) significantly outperformed Pipeline A (30.73 CodeBLEU). At this highly constrained scale (50K methods, 3 epochs), the pre-training objective likely interfered with the downstream task. Pipeline B’s random initialization allowed it to converge more directly on the specific bug-fixing objective, whereas Pipeline A had to ”unlearn” its span-corruption weights.

5.3 The ”Babbling” Inference Anomaly

Despite Pipeline B’s superior CodeBLEU, we observed a massive difference in inference efficiency. Pipeline A evaluated the test set in 10 minutes, whereas Pipeline B took over 100 minutes (**10.5x slower**). Because Pipeline B lacked the structural foundation of pre-training, it failed to learn sequence boundaries (the `</s>` token). Consequently, beam search forced the model to ”babble” up to the `max_new_tokens=256` limit for many samples, resulting in a much larger inference time.

5.4 Task-Specific Models vs. RAG

Qwen 1.5B achieved a Zero-Shot CodeBLEU of 42.84. Integrating a 3-Shot RAG approach pushed this to 44.51, validating that semantic context helps guide general models. However, the specialized Pipeline B still yielded the highest similarity score of 51.99.

6 Conclusion

This study demonstrates a clear deployment trade-off. Pre-training is essential for teaching small models structural constraints, preventing the ”babbling” anomaly and ensuring high inference speed. While scratch-training can maximize raw similarity metrics, it results in a computationally inefficient model. Domain-specific pre-training remains the most robust strategy for deploying high-efficiency software tools in real-time environments.